

5. Facade Pattern.

Provide a simplified interface to a complex subsystem.

In simple terms:

- Hide complex system interactions.
- Provide a single easy entry point.

Eg:- Starting a computer main module.

without Facade

- CPU
- memory
- Hard Drive
- OS.

→ CPU.start()
memory.load()
Hard Drive.read()
OS.initialize()

Client must know too many details.

Eg:- Instead of contacting:

House Keeping, Restaurant, Laundry and Taxi,
you contact Reception. Reception ⇒ Facade.

Code

1. Subsystem Classes.

```
class CPU {  
    void start() { S.O.pln("CPU started"); }  
}
```

```
class memory {  
    void load() { S.O.pln("Memory loaded"); }  
}
```

```
class HardDrive {  
    void read() { S.O.pln("Hard drive reading"); }  
}
```


2. Facade Class

```
class ComputerFacade {  
    private CPU cpu;  
    private Memory memory;  
    private HardDrive hardDrive;
```

```
    ComputerFacade() {  
        cpu = new CPU();  
        memory = new Memory();  
        hardDrive = new HardDrive();  
    }
```

```
    void startComputer() {  
        cpu.start();  
        memory.load();  
        hardDrive.read();  
    }  
}
```

3. Client code

```
ComputerFacade computer = new ComputerFacade();  
computer.startComputer();
```

- Client sees simple interface.
- Complexity hidden.

Real world use cases

- Spring Framework APIs
- Logging Frameworks.
- DB access layers.
- Payment gateways.

PaymentService.processPayment() internally calls:

- Fraud Check()
- Balance Check()
- Debit

notification()